

Name:P.Tejaswini
Rollno:2520030011

Program:

```
tejaswini@LAPTOP-S8US186U × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

typedef struct {
    char *value;
} Token;

typedef struct {
    Token tokens[MAX_TOKENS];
    int count;
} TokenList;

// Check whether character is a delimiter
int is_delimiter(char c) {
    return c == '|' || c == '>' || c == '<' || c == ';';
}

// Tokenize input
void tokenize(char *input, TokenList *list) {
    list->count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && list->count < MAX_TOKENS) {
        list->tokens[list->count].value = malloc(strlen(token) + 1);

        if (list->tokens[list->count].value == NULL) {
            printf("Memory allocation failed\\n");
            exit(1);
        }

        strcpy(list->tokens[list->count].value, token);

        list->count++;

        token = strtok(NULL, " \\t\\n");
    }
}
```



tejaswini@LAPTOP-S8US186U



GNU nano 8.7.1

```
}  
  
// Display tokens  
void display_tokens(TokenList *list) {  
  
    printf("\nTokens:\n");  
  
    for (int i = 0; i < list->count; i++) {  
        printf("Token %d: %s\n",  
               i + 1,  
               list->tokens[i].value);  
    }  
}  
  
// Validate token stream  
int validate_tokens(TokenList *list) {  
  
    if (list->count == 0) {  
        printf("Empty command\n");  
        return 0;  
    }  
  
    // Command cannot start with pipe  
    if (strcmp(list->tokens[0].value, "|") == 0) {  
        printf("Syntax Error: command cannot start with '|' \n");  
        return 0;  
    }  
  
    // Command cannot end with pipe  
    if (strcmp(list->tokens[list->count - 1].value, "|") == 0) {  
        printf("Syntax Error: command cannot end with '|' \n");  
        return 0;  
    }  
  
    // Two consecutive pipes are invalid  
    for (int i = 0; i < list->count - 1; i++) {  
  
        if (strcmp(list->tokens[i].value, "|") == 0 &&  
            strcmp(list->tokens[i + 1].value, "|") == 0) {  
  
            printf("Syntax Error: consecutive pipes\n");  
            return 0;  
        }  
    }  
}
```



tejaswini@LAPTOP-S8US186U



GNU nano 8.7.1

```
// Simple parse tree representation
void create_parse_tree(TokenList *list) {

    printf("\nParse Tree:\n");

    printf("COMMAND\n");

    for (int i = 0; i < list->count; i++) {

        if (strcmp(list->tokens[i].value, "|") == 0) {
            printf(" PIPE\n");
        }
        else if (strcmp(list->tokens[i].value, ">") == 0) {
            printf(" OUTPUT_REDIRECTION\n");
        }
        else if (strcmp(list->tokens[i].value, "<") == 0) {
            printf(" INPUT_REDIRECTION\n");
        }
        else {
            printf(" ARGUMENT: %s\n",
                list->tokens[i].value);
        }
    }
}

// Free allocated memory
void free_tokens(TokenList *list) {

    for (int i = 0; i < list->count; i++) {
        free(list->tokens[i].value);
    }

    list->count = 0;
}

int main() {

    char input[MAX_INPUT];

    TokenList list;

    printf("Enter a command: ");

    fgets(input, sizeof(input), stdin);
```

```

// Remove newline
input[strcspn(input, "\n")] = '\0';

// Handle empty command
if (strlen(input) == 0) {
    printf("Empty command\n");
    return 0;
}

tokenize(input, &list);

display_tokens(&list);

if (validate_tokens(&list)) {
    printf("\nSyntax is valid.\n");

    create_parse_tree(&list);

    printf("\nExecution structure created successfully.\n");
}
else {
    printf("\nInvalid command.\n");
}

free_tokens(&list);

return 0;
}
|

```

Output:

```

tejaswini@LAPTOP-S8US186U:~$ gcc skill_4.c -o skill_4
tejaswini@LAPTOP-S8US186U:~$ ./skill_4
Enter a command: ls -l

Tokens:
Token 1: ls
Token 2: -l

Syntax is valid.

Parse Tree:
COMMAND
  ARGUMENT: ls
  ARGUMENT: -l

Execution structure created successfully.

```

```
tejaswini@LAPTOP-S8US186U:~$ gcc skill_4.c -o skill_4
tejaswini@LAPTOP-S8US186U:~$ ./skill_4
Enter a command: ls -l | grep txt
```

Tokens:

Token 1: ls
Token 2: -l
Token 3: |
Token 4: grep
Token 5: txt

Syntax is valid.

Parse Tree:

COMMAND

ARGUMENT: ls
ARGUMENT: -l
PIPE
ARGUMENT: grep
ARGUMENT: txt

Execution structure created successfully.

```
tejaswini@LAPTOP-S8US186U:~$ ./skill_4
Enter a command: ls || grep
```

Tokens:

Token 1: ls
Token 2: ||
Token 3: grep

Syntax is valid.

Parse Tree:

COMMAND

ARGUMENT: ls
ARGUMENT: ||
ARGUMENT: grep

Execution structure created successfully.

```
tejaswini@LAPTOP-S8US186U:~$ |
```